Objetivos:

- Compreender como as telas são organizadas na pilha de navegação do Flutter.
- Utilizar Navigator.push() e Navigator.pop() para avançar e voltar à tela anterior.
- Enviar dados para outra tela por meio do construtor de um widget.

Indice:

i. Projeto base da aula
ii. Navigator
iii. Envio de dados para outra tela

i. Projeto base da aula

O projeto terá três telas: Início, Detalhes e Confirmação. A navegação ocorre nessa ordem, formando uma pilha.

Na tela de Confirmação, o usuário pode voltar para Detalhes ou retornar diretamente ao Início.

O nome e data de nascimento informado no Início é enviado para as outras duas telas.

Crie um projeto usando, por exemplo

```
flutter create app --platform web, android
```

Substitua o conteúdo de `lib/main.dart` por:

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const RootApp());
}

String formatarData(DateTime data) {
  final dia = data.day.toString().padLeft(2, '0');
  final mes = data.month.toString().padLeft(2, '0');

  return '$dia/$mes/${data.year}';
}

class RootApp extends StatelessWidget {
  const RootApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Navegação entre telas',
      home: TelaInicio(),
    );
  }
}
```

```dart
class TelaInicio extends StatefulWidget {
  const TelaInicio({super.key});

  @override
  State<TelaInicio> createState() => _TelaInicioState();
}

class _TelaInicioState extends State<TelaInicio> {
  final TextEditingController controladorNome = TextEditingController();
  DateTime? dataNascimento;

  Future<void> selecionarData() async {
    final dataSelecionada = await showDatePicker(
      context: context,
      initialDate: dataNascimento ?? DateTime(2000),
      firstDate: DateTime(1900),
      lastDate: DateTime.now(),
    );

    if (!mounted) return;

    if (dataSelecionada != null) {
      setState(() {
        dataNascimento = dataSelecionada;
      });
    }
  }

  void abrirDetalhes() {
    final nome = controladorNome.text.trim();
    final data = dataNascimento;

    if (nome.isEmpty || data == null) return;

    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => TelaDetalhes(nome: nome, dataNascimento: data),
      ),
    );
  }

  @override
  void dispose() {
    controladorNome.dispose();
    super.dispose();
  }
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Início')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            TextField(
              controller: controladorNome,
              decoration: const InputDecoration(labelText: 'Nome'),
            ),
            const SizedBox(height: 16),
            Text(
              dataNascimento == null
                  ? 'Data de nascimento não selecionada'
                  : 'Nascimento: ${formatarData(dataNascimento!)}',
            ),
            ElevatedButton(
              onPressed: selecionarData,
              child: const Text('Selecionar data de nascimento'),
            ),
            const SizedBox(height: 16),
            ElevatedButton(
              onPressed: abrirDetalhes,
              child: const Text('Abrir detalhes'),
            ),
          ],
        ),
      ),
    );
  }
}

class TelaDetalhes extends StatelessWidget {
  final String nome;
  final DateTime dataNascimento;

  const TelaDetalhes({
    super.key,
    required this.nome,
    required this.dataNascimento,
  });

  int calcularIdade(DateTime hoje) {
    var idade = hoje.year - dataNascimento.year;

    final aindaNaoFezAniversario =
```

```
      hoje.month < dataNascimento.month ||
      (hoje.month == dataNascimento.month && hoje.day < dataNascimento.day);

  if (aindaNaoFezAniversario) {
    idade--;
  }

  return idade;
}

@override
Widget build(BuildContext context) {
  final idade = calcularIdade(DateTime.now());
  final maiorDeIdade = idade >= 18;

  return Scaffold(
    appBar: AppBar(title: const Text('Detalhes')),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text('Nome: $nome'),
          Text('Nascimento: ${formatarData(dataNascimento)}'),
          Text('Idade: $idade anos'),
          const SizedBox(height: 16),
          Text(
            maiorDeIdade
                ? 'A pessoa tem 18 anos ou mais.'
                : 'A pessoa tem menos de 18 anos.',
          ),
          const SizedBox(height: 16),
          if (maiorDeIdade)
            ElevatedButton(
              onPressed: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (context) => TelaConfirmacao(nome: nome),
                  ),
                );
              },
              child: const Text('Abrir confirmação'),
            ),
          ElevatedButton(
            onPressed: () => Navigator.pop(context),
            child: const Text('Voltar ao início'),
          ),
        ],
```

```
        ),
      ),
    );
  }
}

class TelaConfirmacao extends StatelessWidget {
  final String nome;

  const TelaConfirmacao({super.key, required this.nome});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Confirmação')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text('Dados de $nome confirmados.'),
            const SizedBox(height: 16),
            ElevatedButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Voltar aos detalhes'),
            ),
            ElevatedButton(
              onPressed: () {
                Navigator.popUntil(context, (rota) => rota.isFirst);
              },
              child: const Text('Voltar ao início'),
            ),
          ],
        ),
      ),
    );
  }
}
```

ii. Navigator

2.1. A pilha de telas

O Navigator organiza a navegação por meio de uma **pilha de rotas**. Quando uma tela é aberta, sua rota é colocada no topo da pilha. Quando o usuário volta, a rota do topo é removida.

No projeto base, o fluxo pode chegar a esta situação:

Posição na pilha	Tela

Posição na pilha	Tela
Base	Início
Meio	Detalhes
Topo	Confirmação

Caso a tela visível seja **Confirmação**, pois sua rota está no topo. Se essa rota for removida, **Detalhes** volta a aparecer.

2.2. Abrindo uma tela com `Navigator.push()`

Na tela Início, o método `abrirDetalhes()` contém a navegação para a próxima tela:

```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => TelaDetalhes(nome: nome, dataNascimento: data),
  ),
);
```

Os elementos principais são:

- `Navigator.push()`: adiciona uma rota ao topo da pilha.
- `context`: permite acessar o `Navigator` associado àquela parte da aplicação.
- `MaterialPageRoute`: cria uma rota que apresenta uma tela seguindo o comportamento visual do Material Design.
- `builder`: informa qual widget será construído para a nova tela.

Depois dessa chamada, a pilha contém Início → Detalhes, e a tela Detalhes fica visível.

O mesmo procedimento é usado para abrir a tela Confirmação a partir de Detalhes:

```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => TelaConfirmacao(nome: nome),
  ),
);
```

Nesse momento, a pilha contém Início → Detalhes → Confirmação.

`push()` abre uma nova rota; ele não substitui a rota anterior. Por isso, é possível voltar à tela que estava aberta.

2.3. Voltando à tela anterior com `Navigator.pop()`

Na tela Confirmação, o botão **Voltar aos detalhes** executa:

```
Navigator.pop(context)
```

Essa instrução remove a rota atual do topo da pilha. A tela Detalhes, que estava logo abaixo, volta a aparecer.

O botão **Voltar ao início**, presente na tela Detalhes, também usa `pop()`:

```
Navigator.pop(context)
```

Nesse caso, a rota removida é Detalhes. Como Início passa a ser o topo da pilha, ela volta a aparecer. Portanto, `pop()` significa **voltar uma tela na pilha**, independentemente do nome dessa tela.

Quando uma nova rota é aberta com `MaterialPageRoute`, o `AppBar` normalmente também apresenta um botão de voltar. Esse botão produz o mesmo efeito de remover a rota atual.

2.4. Voltando diretamente ao início com `Navigator.popUntil()`

Na tela Confirmação, voltar ao Início exige retirar **duas** rotas da pilha: Confirmação e Detalhes. Para isso, o projeto usa:

```
Navigator.popUntil(context, (rota) => rota.isFirst)
```

`popUntil()` remove rotas até encontrar uma que satisfaça a condição informada. A expressão `rota.isFirst` identifica a primeira rota da pilha, que neste projeto corresponde à tela Início.

iii. Envio de dados para outra tela

3.1. Introdução

No projeto base, a tela **Início** recebe o nome e a data de nascimento de uma pessoa. A tela **Detalhes** precisa desses dados para apresentá-los e verificar se a pessoa tem pelo menos 18 anos. Depois, o nome também é enviado à tela **Confirmação**.

Neste capítulo, veremos como enviar dados **da tela que abre uma rota para a nova tela**, utilizando o construtor do widget.

3.2. Dados recebidos pelo construtor

Para receber o nome e a data de nascimento, `TelaDetalhes` declara dois campos e os exige em seu construtor:

```
class TelaDetalhes extends StatelessWidget {
  final String nome;
  final DateTime dataNascimento;

  const TelaDetalhes({
    super.key,
    required this.nome,
    required this.dataNascimento,
  });

  // ...
}
```

A palavra `required` obriga quem cria `TelaDetalhes` a fornecer os dois valores. Os campos são `final` porque os dados recebidos pelo widget não devem ser alterados diretamente.

Assim, a tela Detalhes dispõe dos dados necessários desde a sua criação.

3.3. Envio dos dados pela tela Início

Antes de navegar, `TelaInicio` lê o texto digitado e verifica se a data foi selecionada:

```
void abrirDetalhes() {
  final nome = controladorNome.text.trim();
  final data = dataNascimento;

  if (nome.isEmpty || data == null) return;

  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (context) => TelaDetalhes(nome: nome, dataNascimento: data),
    ),
  );
}
```

A comunicação acontece na construção de `TelaDetalhes`:

```
TelaDetalhes(nome: nome, dataNascimento: data)
```

O valor à **esquerda** dos dois-pontos identifica o parâmetro do construtor. O valor à **direita** é o dado fornecido pela tela Início.

A verificação anterior ao `Navigator.push()` garante que a tela Detalhes só seja criada quando os dois valores estiverem disponíveis.

3.4. Envio do nome para a terceira tela

A tela Detalhes também abre a tela Confirmação. Nesse caso, apenas o nome é necessário:

```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => TelaConfirmacao(nome: nome),
  ),
);
```

`TelaConfirmacao` recebe o valor da mesma maneira:

```
class TelaConfirmacao extends StatelessWidget {
  final String nome;

  const TelaConfirmacao({super.key, required this.nome});
```

```
  // ....
}
```

iv. Retorno de dados à tela anterior

4.1. Introdução

No capítulo anterior, enviamos dados de Detalhes para Confirmação pelo construtor de `TelaConfirmacao`. Agora faremos a comunicação no sentido inverso: a tela Confirmação devolverá um resultado à tela Detalhes.

O resultado será um bool:

- true: o usuário confirmou os dados;
- null: o usuário voltou sem confirmar.

A apresentação desse resultado na interface será abordada no capítulo 5.

4.2. Retornando um valor com `Navigator.pop()`

Até agora, usamos `Navigator.pop(context)` apenas para fechar a tela atual. O método também aceita um **resultado** como segundo argumento:

```
Navigator.pop(context, true);
```

Essa instrução fecha a tela Confirmação e devolve true para a tela que a abriu.

Na `TelaConfirmacao` do projeto base, inclua o botão de confirmação antes dos botões de retorno:

```
ElevatedButton(
  onPressed: () => Navigator.pop(context, true),
  child: const Text('Confirmar dados'),
),
```

O botão **Voltar aos detalhes** continua sem fornecer resultado:

```
ElevatedButton(
  onPressed: () => Navigator.pop(context),
  child: const Text('Voltar aos detalhes'),
),
```

4.3. Recebendo o resultado de `Navigator.push()`

`Navigator.push()` retorna um Future. Esse Future é concluído quando a tela aberta é fechada. Para receber o valor enviado por `pop()`, utilizamos `await`.

Na classe `TelaDetalhes`, crie o seguinte método:

```
Future<void> abrirConfirmacao(BuildContext context) async {
  final resultado = await Navigator.push<bool>(
    context,
    MaterialPageRoute(builder: (context) => TelaConfirmacao(nome: nome)),
  );
```

```
  if (resultado == true) {
    debugPrint('Os dados de $nome foram confirmados.');
  }
}
```

O tipo <bool> indica que esperamos um resultado booleano. Entretanto, a variável resultado pode ser null, pois o usuário pode voltar sem tocar em **Confirmar dados**. Por esse motivo, verificamos `resultado == true`.

Nesse momento, `debugPrint()` apenas permite observar o valor recebido no console. No próximo capítulo, utilizaremos o resultado para atualizar a tela Detalhes.

Ainda em `TelaDetalhes`, substitua o `onPressed` do botão **Abrir confirmação**:

```
if (maiorDeIdade)
  ElevatedButton(
    onPressed: () => abrirConfirmacao(context),
    child: const Text('Abrir confirmação'),
  ),
```

A sequência de execução passa a ser:

1. O usuário toca em **Abrir confirmação**.
2. `Navigator.push<bool>()` abre a tela Confirmação;
3. A execução de `abrirConfirmacao()` aguarda em `await`;
4. O usuário toca em **Confirmar dados**;
5. `Navigator.pop(context, true)` fecha a tela e devolve `true`;
6. A execução de `abrirConfirmacao()` continua com `resultado == true`.

Se o usuário tocar em **Voltar aos detalhes** ou usar o botão de voltar do AppBar, a tela Confirmação será fechada sem enviar `true`. Nesse caso, resultado será `null` e a condição não será satisfeita:

```
if (resultado == true) {
  debugPrint('Os dados de $nome foram confirmados.');
}
```

Exercício

Exercício – Aplicativo de pedidos de restaurante

Desenvolva um aplicativo Flutter que permita ao usuário montar um pedido em cinco telas. Utilize como referência o exemplo de navegação disponível no projeto base e implemente a solução no arquivo um.dart.

Telas da aplicação:

1. Prato principal: apresentar pelo menos três opções com seus respectivos preços e permitir selecionar uma delas.
2. Bebida: apresentar pelo menos três opções com seus respectivos preços e permitir selecionar uma delas.

3. Sobremesa: apresentar pelo menos três opções com seus respectivos preços e permitir selecionar uma delas.
4. Forma de pagamento: permitir escolher entre Pix, cartão de crédito, cartão de débito e dinheiro.
5. Finalização do pedido: exibir os itens escolhidos, o preço de cada item, o valor total e a forma de pagamento.

Regras de funcionamento:

- O usuário só poderá avançar após selecionar uma opção na tela atual.
- Nas telas 2, 3 e 4, deverá ser possível voltar à tela anterior ou diretamente ao início.
- Ao voltar à tela anterior, a escolha feita nessa tela deverá permanecer selecionada e poderá ser alterada.
- Na tela de finalização, não será permitido retornar às etapas anteriores. A única ação disponível será "Novo pedido", que levará à primeira tela com as escolhas reiniciadas.
- Os valores deverão ser apresentados em reais, com duas casas decimais.

Requisitos técnicos:

- Utilize Navigator e MaterialPageRoute para navegar entre as telas.
- Passe os dados necessários entre as telas por meio dos construtores.
- Utilize StatefulWidget e setState para controlar as seleções.
- Defina os itens do cardápio e seus preços no próprio código.

Organize o projeto usando a seguinte estrutura:

```
lib/
├── main.dart
├── models/
│   └── item_cardapio.dart
├── utils/
│   └── formatar_valor.dart
└── telas/
    ├── tela_prato_principal.dart
    ├── tela_bebida.dart
    ├── tela_sobremesa.dart
    ├── tela_pagamento.dart
    └── tela_finalizacao.dart
```

Conteúdo do arquivo `models/item_cardapio.dart`:

```dart
class ItemCardapio {
  final String nome;
  final double valor;

  const ItemCardapio(this.nome, this.valor);
}
```

Conteúdo do arquivo `utils/formatar_valor.dart`:

```
String formatarValor(double valor) {
  return 'R\$ ${valor.toStringAsFixed(2).replaceAll('.', ',')}';
}
```